

Data Structures Lab: Binary Tree Creation and Traversals

Lab Experiment

Experiment No.: ____

Title: Creation of a Binary Tree and Tree Traversals

Aim

To write a C program to create a **binary tree** and perform the following traversals:

1. Pre-order traversal
2. In-order traversal
3. Post-order traversal

Learning Objectives

After completing this experiment, students should be able to:

- Understand the structure of a binary tree.
- Create nodes dynamically using pointers.
- Construct a binary tree recursively.
- Implement pre-order, in-order, and post-order traversals.
- Understand the difference between the three traversal techniques.

Theory

A **binary tree** is a tree data structure in which each node can have at most two children:

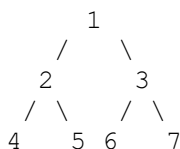
- **Left child**
- **Right child**

A binary tree node can be represented in C as:

```
struct Node {  
    int data;  
    struct Node *left;  
    struct Node *right;  
};
```

Tree Traversals

For the following binary tree:



1. Pre-order Traversal

Order:

Root → Left → Right

Output:

1 2 4 5 3 6 7

2. In-order Traversal

Order:

Left → Root → Right

Output:

4 2 5 1 6 3 7

3. Post-order Traversal

Order:

Left → Right → Root

Output:

4 5 2 6 7 3 1

C Program

```
#include <stdio.h>
#include <stdlib.h>

// Structure of a binary tree node
struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
}
```

```

        newNode->data = data;
        newNode->left = NULL;
        newNode->right = NULL;

        return newNode;
    }

    // Pre-order traversal
    void preorder(struct Node *root) {
        if (root != NULL) {
            printf("%d ", root->data);
            preorder(root->left);
            preorder(root->right);
        }
    }

    // In-order traversal
    void inorder(struct Node *root) {
        if (root != NULL) {
            inorder(root->left);
            printf("%d ", root->data);
            inorder(root->right);
        }
    }

    // Post-order traversal
    void postorder(struct Node *root) {
        if (root != NULL) {
            postorder(root->left);
            postorder(root->right);
            printf("%d ", root->data);
        }
    }

    // Main function
    int main() {

        // Creating the binary tree
        struct Node *root = createNode(1);

        root->left = createNode(2);
        root->right = createNode(3);

        root->left->left = createNode(4);
        root->left->right = createNode(5);

        root->right->left = createNode(6);
        root->right->right = createNode(7);

        // Display traversals
        printf("Pre-order Traversal : ");
        preorder(root);

        printf("\nIn-order Traversal  : ");
        inorder(root);

        printf("\nPost-order Traversal : ");
        postorder(root);

        printf("\n");
    }

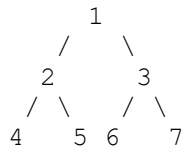
```

```
        return 0;
    }
```

Sample Output

```
Pre-order Traversal : 1 2 4 5 3 6 7
In-order Traversal  : 4 2 5 1 6 3 7
Post-order Traversal : 4 5 2 6 7 3 1
```

Tree Used in the Program



Algorithm

Pre-order:

```
PREORDER(root)
1. If root is NULL, return.
2. Visit root.
3. PREORDER(root->left).
4. PREORDER(root->right).
```

In-order:

```
INORDER(root)
1. If root is NULL, return.
2. INORDER(root->left).
3. Visit root.
4. INORDER(root->right).
```

Post-order:

```
POSTORDER(root)
1. If root is NULL, return.
2. POSTORDER(root->left).
3. POSTORDER(root->right).
4. Visit root.
```

Lab Questions

1. What is a binary tree?
2. What is the maximum number of children of a node in a binary tree?
3. What is pre-order traversal?
4. What is in-order traversal?
5. What is post-order traversal?
6. What is the difference between a binary tree and a binary search tree?
7. Why is recursion commonly used for tree traversal?
8. What is the time complexity of tree traversal?
9. What happens when the root pointer is NULL?

10. Draw a binary tree with 7 nodes and find its pre-order, in-order, and post-order traversals.